

Distributed Job Scheduler — Architecture & Design Document

Phase 0 Deliverable

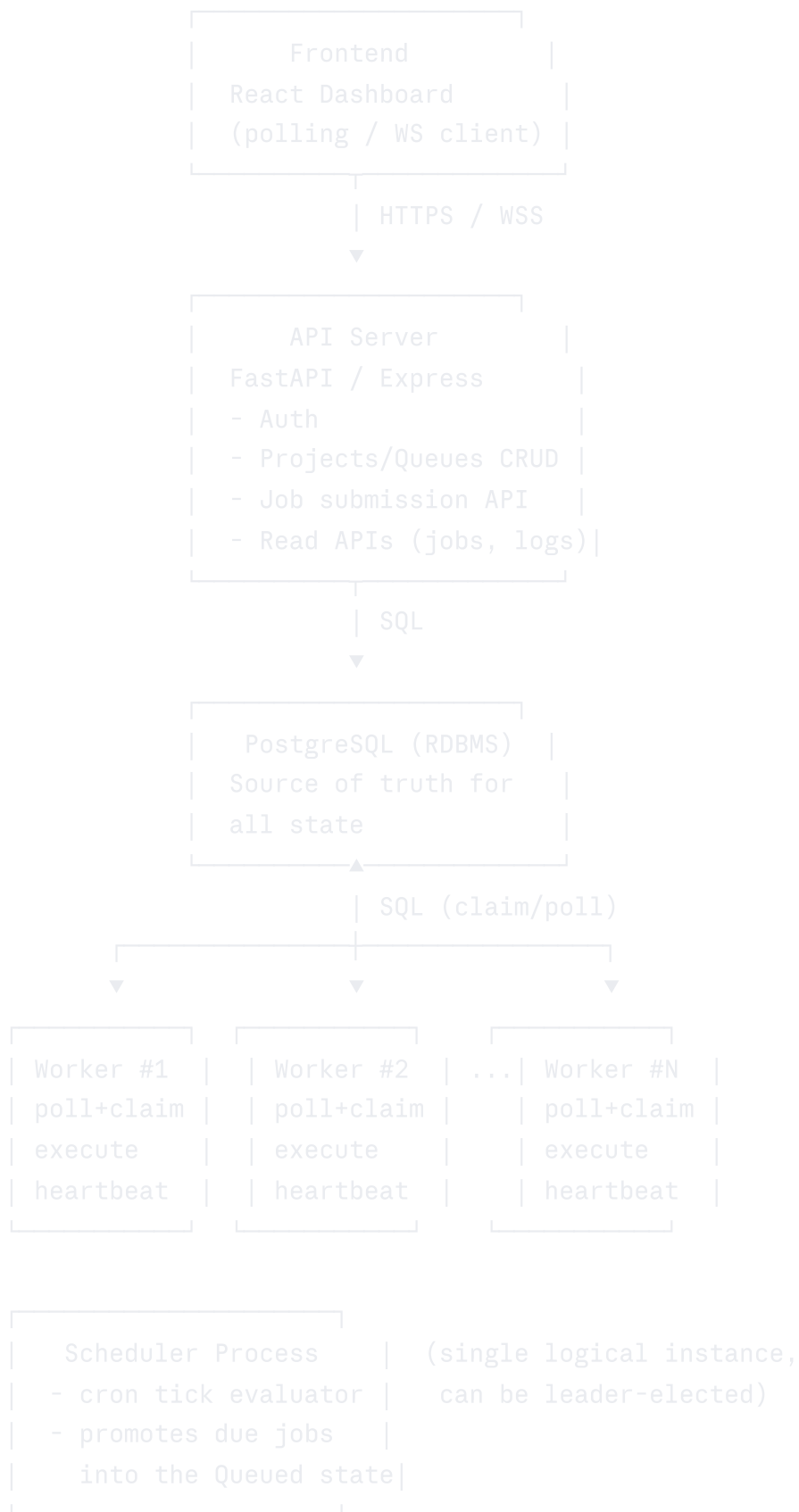
1. Problem Restatement

Build a system where users define **queues**, submit **jobs** to those queues (immediate, delayed, cron-scheduled, or batch), and a fleet of **worker processes** picks up jobs, executes them concurrently, and reports results — with retries, dead-lettering, and full observability — without ever letting two workers execute the same job instance at the same time.

The core engineering risk is **race conditions on job claiming**. Everything else in the spec is built on top of getting that right.

2. High-Level System Architecture

Four independent, horizontally-scalable components, talking only through the database (and optionally a lightweight pub/sub channel for live updates):



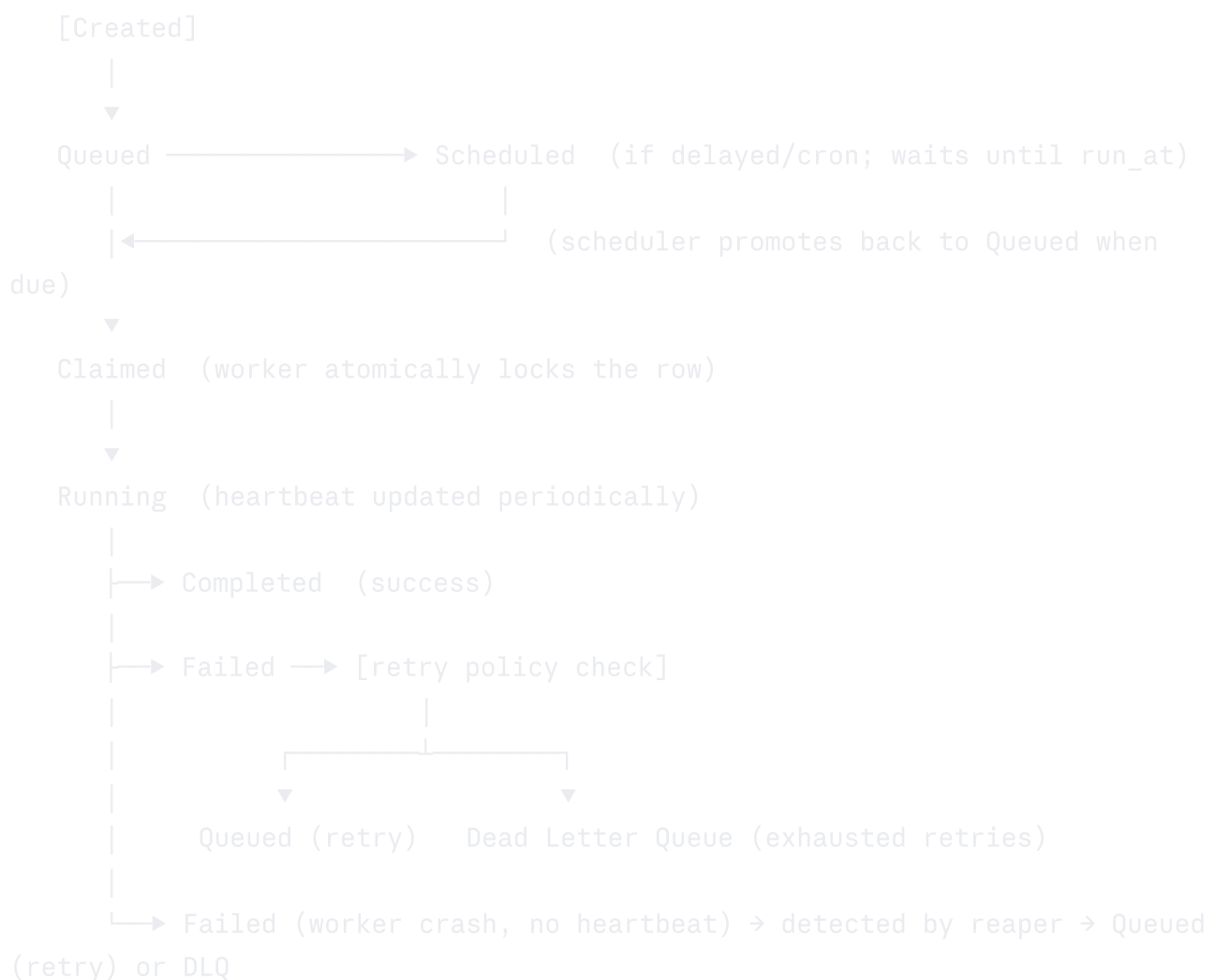
Optional: Redis for pub/sub (WebSocket fan-out) and/or distributed locks for scheduler leader election.

Why four components and not one monolith:

- The **API server** is stateless and only ever talks to Postgres — it can be scaled behind a load balancer trivially.

- **Workers** are a separate deployable unit because their scaling dimension is different (CPU/IO bound by job execution, not HTTP traffic).
 - The **Scheduler** is logically separate because "what jobs are due right now" is a different responsibility than "execute a job" — conflating them makes cron logic leak into every worker.
 - All coordination goes through **Postgres**, not through a message broker, because Postgres row-locking gives us atomicity guarantees for free and keeps the system simpler to build and grade in an internship timeframe. (A Redis/Kafka queue is a valid alternative — documented as a trade-off in §6.)
-

3. Job Lifecycle — State Machine



Recurring (cron) jobs are template rows: the scheduler generates a **new job instance** each time the cron expression fires, rather than reusing one row — this keeps execution history per-run and matches how Sidekiq/Airflow do it.

4. Atomic Job Claiming — The Core Concurrency Mechanism

This is the single most important design decision in the whole system and the biggest source of marks under "Reliability & Concurrency."

Chosen approach: `SELECT ... FOR UPDATE SKIP LOCKED`

```
sql

UPDATE jobs
SET status = 'claimed', worker_id = :worker_id, claimed_at = now()
WHERE id = (
    SELECT id FROM jobs
    WHERE status = 'queued'
      AND queue_id = :queue_id
      AND run_at <= now()
    ORDER BY priority DESC, run_at ASC
    FOR UPDATE SKIP LOCKED
    LIMIT :batch_size
)
RETURNING *;
```

Why this beats the alternatives:

Approach	Problem
Plain <code>SELECT</code> then <code>UPDATE</code>	Race condition — two workers can select the same row before either updates it
<code>SELECT ... FOR UPDATE</code> (no <code>SKIP LOCKED</code>)	Workers queue up and block on each other's row locks — kills throughput
Optimistic locking (version column + CAS)	Works, but wastes cycles on retry storms under high contention; more code for equivalent safety
<code>FOR UPDATE SKIP LOCKED</code>	Each worker instantly skips rows another worker already has locked — no blocking, no duplicate claims, single round trip

This single SQL pattern is what makes concurrency-safe polling possible without a message broker. It should be called out explicitly in your design-decisions document — graders will look for it specifically.

Crash detection (the reaper): a background check (in the scheduler process) finds jobs stuck in `Running` whose last heartbeat exceeds a timeout, and requeues or dead-letters them. This is what makes the system resilient to worker crashes, not just worker failures.

5. Retry Strategy Design

Each queue (or job) carries a `retry_policy`:

```
json
{
  "strategy": "exponential",    // fixed | linear | exponential
  "base_delay_seconds": 5,
  "max_retries": 5,
  "max_delay_seconds": 3600
}
```

- **Fixed:** `delay = base_delay_seconds`
- **Linear:** `delay = base_delay_seconds * attempt_number`
- **Exponential:** `delay = min(base_delay_seconds * 2^attempt_number, max_delay_seconds)` — add jitter to avoid thundering-herd retries

On exhausting `max_retries`, the job moves to the **Dead Letter Queue** table (not deleted) so it can be inspected and manually replayed from the dashboard.

6. Key Design Trade-offs (for your design-decisions doc)

Decision	Alternative considered	Why this choice
Postgres as the only broker (no Redis/Kafka queue)	Redis-backed queue (BullMQ-style) or Kafka	Fewer moving parts, transactional consistency between job state and business data, <code>SKIP LOCKED</code> gives real atomicity. Trade-off: lower theoretical max throughput than a dedicated broker — acceptable for this scope.
Polling workers vs. push-based dispatch	Server pushes jobs to workers over gRPC/WebSocket	Polling is simpler, self-healing (a dead dispatcher can't strand workers), and naturally load-balances. Trade-off: added latency = poll interval.
Separate Scheduler process	Cron logic embedded in each worker	Avoids N workers all evaluating the same cron expressions redundantly and racing to promote the same due job.
Heartbeats stored in DB vs. separate service (e.g. etcd)	Dedicated liveness service	Keeps infra minimal; Postgres row updates are cheap enough at this scale.

Decision	Alternative considered	Why this choice
Idempotency via job-level idempotency key	None / rely on at-most-once claiming	Claiming prevents duplicate <i>concurrent</i> execution, but a retried job after a crash could re-run side effects — an idempotency key lets job handlers de-dupe safely.

7. Recommended Tech Stack

Layer	Choice	Why
API Server	FastAPI (Python) or Express/Nest (TypeScript)	Async I/O, good Postgres drivers, fast to build validated REST APIs
Database	PostgreSQL	Row-level locking, <code>SKIP LOCKED</code> , strong consistency — this is the backbone of the whole design
Worker	Same language as API, separate process/container	Shares ORM models/migrations, simplifies development
Frontend	React + Vite + Tailwind	Matches your existing stack experience (ROS 2.0), fast to iterate
Live updates	WebSockets (or polling fallback)	Dashboard requirement; Redis pub/sub if you scale to multiple API instances
Containerization	Docker Compose (api, worker×N, scheduler, postgres, redis, frontend)	Matches assignment's "distributed" framing, easy to demo locally

8. What Phase 0 Deliverables This Covers

- ✓ Architecture diagram (§2)
- ✓ Job lifecycle / state machine (§3)
- ✓ Core reliability mechanism explained (§4)
- ✓ Retry design (§5)
- ✓ Design trade-offs document seed (§6)
- ✖ ER diagram — **next**, in Phase 0b, since it deserves its own detailed pass (11 entities, FK/index reasoning)

Next Step

Phase 0 is architecture-only — no code yet. The next piece is the **database schema (ER diagram + full DDL reasoning)** covering Users, Organizations, Projects, Queues, Jobs, Job Executions, Retry Policies, Workers, Worker Heartbeats, Job Logs, Scheduled Jobs, and DLQ entries, with indexing strategy explained.

Want me to go straight into that (Phase 0b — DB design), or do you want to sit with this architecture first and poke holes in it?